

Bayesian Machine Learning in Quantitative Finance

Chapter 5: Option Prices via Normalizing Flows

Based on Mongwe, Mbuva & Marwala

Chapter Outline

- 1 5.1 Introduction
- 2 5.2 Risk-Neutral Density from Option Prices
- 3 5.3 Normalizing Flows
- 4 5.4 Numerical Experiments
- 5 5.5 Results and Discussions
- 6 5.6 Conclusions

5.1 Introduction: Why This Chapter? I

The Problem With Black–Scholes

Geometric Brownian motion (the engine behind Black–Scholes) assumes log-returns are Gaussian. Real equity returns are not: they show **fat tails** and a **volatility skew** across strikes. Two classical fixes exist:

- let volatility itself be stochastic (Heston model – Chapter 4),
- add jumps to the price process (Merton, Kou jump-diffusion models).

Both are *economically motivated*: you write down assumptions about the world, then derive option prices from them. They are interpretable, but hard to extend – adding parameters can break their economic derivation.

5.1 Introduction: Why This Chapter? II

The Machine-Learning Alternative

Instead of assuming dynamics, treat option pricing as a **regression problem**: fit a flexible function $f_w(K)$ (a neural network, say) directly to observed market prices at strikes K . This can fit market prices almost perfectly, but the fitted curve is *not guaranteed to be arbitrage-free* – it might imply negative probabilities or a negative fair option price somewhere. Custom loss penalties can discourage this, but cannot *guarantee* it.

5.1 Introduction: Why This Chapter? III

This Chapter's Idea

Instead of modelling the price curve $f_w(K)$ directly, model the **risk-neutral density** implied by the prices, and only afterwards *derive* the prices from that density. If the object we learn is guaranteed to be a valid probability density, the option prices we recover from it are *automatically* arbitrage-free – no custom penalty terms needed. The tool used to learn this density is a **normalizing flow**.

What This Chapter Does

- Uses the mixture-of-normalizing-flows framework of Yang & Hospedales (2023), originally applied to a developed market (Apple-style US options),
- applies it instead to European options on the **JSE All Share Index** (South Africa, a developing market),
- compares mixtures with 1, 8, 15, and 30 normalizing-flow components, across five option maturities (14, 20, 30, 60, 90 days).

5.1 Introduction: Why This Chapter? IV

Roadmap for This Chapter

- 1 Sec. 5.2: how a risk-neutral density is implied by option prices, and the no-arbitrage constraints it must satisfy.
- 2 Sec. 5.3: what a normalizing flow is, and how it can represent this density.
- 3 Sec. 5.4: the JSE data and experimental setup.
- 4 Sec. 5.5–5.6: results and conclusions.

5.2 What Is a “Risk-Neutral Density”? I

Plain-Language Definition (read this first)

The risk-neutral density is the probability distribution over the **future asset price** that is *implied by observed option prices* – it is **not** the real-world (physical) distribution of the stock price. It is the distribution consistent with **no-arbitrage pricing**: if you assume the market is arbitrage-free, there must exist some probability measure under which every option’s price equals the discounted expected payoff. The remarkable fact (Breed–Litzenberger, 1978) is that you can **back this density out purely from how call option prices change with strike** – no model of the stock’s dynamics is required.

5.2 What Is a “Risk-Neutral Density”? II

Intuition

A call option with strike K pays $\max(S_T - K, 0)$ at maturity τ . Its price is

$$C(K) = e^{-r\tau} \mathbb{E}^{\mathbb{P}^*}[\max(S_T - K, 0)] = e^{-r\tau} \int_K^{\infty} (S - K) q(S) dS,$$

where $q(S)$ is the risk-neutral density of S_T . Differentiating this integral twice with respect to K removes the payoff structure and leaves $q(K)$ itself (the classical Breeden–Litzenberger relation):

$$q(K) = e^{r\tau} \frac{\partial^2 C(K)}{\partial K^2}.$$

This is exactly *why* a valid, arbitrage-free call-price curve must be **convex** and **decreasing** in K – a probability density cannot be negative.

5.2 What Is a “Risk-Neutral Density”? III

Carr, Geman, Madan & Yor (2003): Static Arbitrage Conditions

Let $f_w(K)$ be the model's fitted option-pricing function at strike K (for a fixed expiry τ), F_τ the forward price, $F_\tau = \exp(-r\tau)S_0$, r the continuously compounded zero-coupon rate, S_0 the initial stock price, and q the continuous dividend yield. Any arbitrage-free pricing curve must satisfy:

$$\frac{\partial f_w(K)}{\partial K} \leq 0 \quad (5.1)$$

$$\frac{\partial^2 f_w(K)}{\partial^2 K} \geq 0 \quad (5.2)$$

$$f_w(\infty) = 0 \quad (5.3)$$

$$\max(0, \exp(-r\tau)(F_\tau - K)) \leq f_w(K) \leq \exp(-r\tau)F_\tau \quad (5.4)$$

Eq. (5.1): prices fall as strike rises. Eq. (5.2): the curve is convex – this is precisely the Breeden–Litzenberger non-negativity of the density above. Eq. (5.3): price $\rightarrow 0$ for very large strikes. Eq. (5.4): the price is bounded between intrinsic value and the discounted forward (as given verbatim in the book).

Python: Toy Option-Price-to-Density Recovery (5.2)

```
1 import numpy as np
2
3 r, tau = 0.05, 0.25          # toy rate, time to expiry
4 S_grid = np.linspace(1.0, 260.0, 20000)  # integration grid for S_T
5
6 def call_price(K, q_density):
7     """Discounted expected payoff under a toy risk-neutral density."""
8     payoff = np.maximum(S_grid - K, 0.0)
9     return np.exp(-r * tau) * np.trapezoid(payoff * q_density, S_grid)
10
11 # Breeden-Litzenberger:  $q(K) = \exp(r \cdot \tau) * d^2 C / d K^2$ 
12 def implied_density(K_grid, prices, r, tau):
13     h = K_grid[1] - K_grid[0]
14     q = np.full_like(prices, np.nan)
15     q[1:-1] = np.exp(r * tau) * (
16         prices[2:] - 2 * prices[1:-1] + prices[:-2]
17     ) / h**2
18     return q
19
20 K_grid = np.linspace(60.0, 150.0, 181)
21 C_vals = np.array([call_price(K, q_true_vals) for K in K_grid])
22 q_recovered = implied_density(K_grid, C_vals, r, tau)
```

5.3 Normalizing Flows: A Recap I

Intuition (recap from Chapter 2)

A normalizing flow builds a complicated target density out of a **simple** base density (usually a standard Gaussian) by pushing samples through a chain of **invertible, differentiable** transformations. Because each step is invertible, we can track exactly how probability mass gets stretched or compressed, and hence compute the exact density of the transformed variable – unlike, say, a GAN, which gives you samples but no tractable density.

5.3 Normalizing Flows: A Recap II

Setup and Notation (as in the book)

Let $x \sim q(x)$ be a sample from a simple **base distribution** (e.g. standard Gaussian). Let T be an invertible, differentiable transformation. Define

$$w = T(x).$$

Here w denotes a sample from the **target (unknown) distribution** $p(w)$ that we want to represent – in this chapter's application, w will play the role of the risk-neutral log-moneyness variable. The **change-of-variables formula** gives the density of w as

$$p(w) = q(x) \left| \det (J_T(x)) \right|^{-1}, \tag{5.9}$$

where $J_T(x)$ is the Jacobian matrix of T evaluated at x . In one dimension this is simply $p(w) = q(x) |T'(x)|^{-1}$.

5.3 Normalizing Flows: A Recap III

Step-by-Step Derivation of Change of Variables (1D, from scratch)

Take a simple invertible map f with $w = f(x)$, so $x = f^{-1}(w)$. We want $p(w)$ given that $x \sim q(x)$.

- 1 Probability is conserved under the map: for any small interval,
 $\mathbb{P}(w \in [f(x), f(x + dx)]) = \mathbb{P}(x \in [x, x + dx])$.
- 2 In terms of densities: $p(w) |dw| = q(x) |dx|$.
- 3 Rearranging: $p(w) = q(x) \left| \frac{dx}{dw} \right| = q(f^{-1}(w)) \left| \frac{df^{-1}(w)}{dw} \right|$.
- 4 Equivalently, since $\frac{dx}{dw} = 1/\frac{dw}{dx}$: $p(w) = q(x) |f'(x)|^{-1}$.

Worked example: let $f(x) = \exp(x)$, so $w = e^x$, $x = \log w$, $f'(x) = e^x = w$. If $x \sim N(0, 1)$, then

$$p(w) = \underbrace{\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(\log w)^2}}_{q(x=\log w)} \cdot \frac{1}{w}, \quad w > 0,$$

which is exactly the **log-normal** density – a normalizing flow with a single exponential layer turns a Gaussian into a log-normal.

Python: A Toy 1D Normalizing Flow From Scratch (5.3)

```
1 import numpy as np
2
3 # Two invertible layers composed: affine, then sinh-arcsinh
4 a, b = 1.0, 0.0          # affine layer
5 eps, delta = 1.0, 0.6   # sinh-arcsinh layer (skew, tail weight)
6
7 def flow_forward(z):
8     u = a * z + b
9     x = np.sinh((np.arcsinh(u) + eps) / delta)
10    return x
11
12 def flow_forward_with_jacobian(z):
13     u, du_dz = a * z + b, a          # layer 1 + its derivative
14     x = np.sinh((np.arcsinh(u) + eps) / delta)  # layer 2
15     dx_du = np.cosh((np.arcsinh(u) + eps) / delta) / (delta * np.sqrt(1 + u**2))
16     return x, dx_du * du_dz          # chain rule -> dx/dz
17
18 def standard_normal_pdf(z):
19     return np.exp(-0.5 * z**2) / np.sqrt(2 * np.pi)
20
21 # Change-of-variables formula:  $p_X(x) = p_Z(z) * |dz/dx| = p_Z(z) / |dx/dz|$ 
22 z_grid = np.linspace(-4.5, 4.5, 4000)
23 x_of_z, dxdz = flow_forward_with_jacobian(z_grid)
24 p_x = standard_normal_pdf(z_grid) / np.abs(dxdz)
```

5.4.1 Data Description I

Intuition

Having defined the model, we now need real market prices to fit it to. The chapter deliberately targets a **developing-market** index (JSE) – previous normalizing-flow work (Yang & Hospedales, Loaiza-Ganem et al.) had only been tested on developed markets (e.g. Apple options).

The Dataset

- European options on the **JSE All Share Index**, expiring **22 July 2019**.
- Five maturities: **14, 20, 30, 60, and 90** days to expiry.
- A subset of the data used in Mongwe et al. (2022).
- Fields available: strikes, the **3-month JIBAR** rate (used as the risk-free proxy), and time to expiry, for both puts and calls.
- **200 call options** in total, split **90/10** into train and test sets.

5.4.1 Data Description II

Why JIBAR and Why This Split Matters

JIBAR (Johannesburg Interbank Average Rate) is the local benchmark rate used in place of a risk-free rate such as LIBOR/SOFR in developed markets – appropriate given the chapter's focus on a South African market. The 90/10 train/test split lets the chapter report genuinely **predictive** performance: the flow mixture is fitted on 90% of strikes and evaluated on the held-out 10%, which is what all reported metrics in Sec. 5.5 describe.

5.4.2 Experiment Setup and Performance Metrics I

Model Configurations Compared

- Mixture of normalizing flows with $M \in \{1, 8, 15, 30\}$ components.
- Each individual normalizing flow has **four layers**.
- Architecture and layer specifications follow Yang & Hospedales (2023) exactly; implementation is in Python, based on the module they provide.
- Training objective: minimize the **mean absolute error** between the market's actual option prices and the model's predicted prices (not a likelihood-only objective).

5.4.2 Experiment Setup and Performance Metrics II

Performance Metrics (evaluated on the held-out test set)

- **MSE** – Mean Squared Error: $\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$, penalizes large errors heavily.
- **MAE** – Mean Absolute Error: $\frac{1}{n} \sum_i |y_i - \hat{y}_i|$, robust to outliers, and also the training loss used here.
- R^2 – coefficient of determination: fraction of variance in market prices explained by the model (closer to 1 is better).
- **MAPE** – Mean Absolute Percentage Error: error scaled by the price level, useful for comparing across strikes of very different price magnitude.

5.4.2 Experiment Setup and Performance Metrics III

What to Watch For in the Results

Because there are five maturities $\times$ four mixture sizes $\times$ four metrics, the natural question when reading Sec. 5.5 is: *does one mixture size dominate across all expiries, or does the best choice depend on time to expiry?* Keep this question in mind – it is exactly what the chapter's headline finding addresses.

5.5 Results: Performance Tables I

Reading the Tables

Tables 5.1–5.5 report R2, MSE, MAPE, and MAE on **unseen (test)** JSE All Share Index option data, for the 14-, 20-, 30-, 60-, and 90-day expiries respectively, across the 1/8/15/30-component mixtures. Bold entries mark the best (outperforming) model for that metric. These are the book's actual reported numbers.

Metric (14-day)	1 mixture	8 mixture	15 mixture	30 mixture
R2	0.99936833	0.99925830	0.99963457	0.99989999
MSE	0.00131217	0.00154073	0.00075910	0.00020792
MAPE	0.14248936	0.17627028	0.14951472	0.07344729
MAE	0.02642767	0.02970133	0.02282180	0.01126697

Table 5.1: 14-day options.

5.5 Results: Performance Tables II

Metric (20-day)	1 mixture	8 mixture	15 mixture	30 mixture
R2	0.99738186	0.99874109	0.99967048	0.99983906
MSE	0.00496526	0.00238749	0.00062491	0.00030521
MAPE	0.29285721	0.11292648	0.05536494	0.03841418
MAE	0.05339091	0.03505068	0.01846331	0.01131606

Table 5.2: 20-day options.

5.5 Results: Performance Tables III

Metric (30-day)	1 mixture	8 mixture	15 mixture	30 mixture
R2	0.99941305	0.99990101	0.99938930	0.99984136
MSE	0.00148616	0.00025062	0.00154610	0.00040166
MAPE	0.08995644	0.02934196	0.03757242	0.02122791
MAE	0.03490084	0.01289784	0.02974536	0.01489130

Table 5.3: 30-day options.

Metric (60-day)	1 mixture	8 mixture	15 mixture	30 mixture
R2	0.99065079	0.99964408	0.99750925	0.99921971
MSE	0.01183464	0.00045052	0.00315288	0.00098771
MAPE	0.11747006	0.01410177	0.07385365	0.02503132
MAE	0.08086322	0.01507402	0.04882987	0.02420844

Table 5.4: 60-day options.

5.5 Results: Performance Tables IV

Metric (90-day)	1 mixture	8 mixture	15 mixture	30 mixture
R2	0.99713960	0.99956541	0.99905050	0.99806153
MSE	0.00518381	0.00078761	0.00172074	0.00351303
MAPE	0.10925920	0.01698751	0.03629014	0.03546318
MAE	0.06111890	0.02207337	0.03577344	0.04499231

Table 5.5: 90-day options.

5.5 Results: Performance Tables V

Headline Finding: Best Mixture Size Depends on Expiry

- For **14-** and **20-day** options: the **30-component** mixture wins on every metric, with the **15-component** mixture a close second.
- For **30-**, **60-**, and **90-day** options: the **8-component** mixture wins on (almost) every metric instead.
- The **single-flow** ($M = 1$) model is, on average, the **worst** performer – confirming that some degree of mixing helps.

Why Does Expiry Matter? – The Kink Argument

As expiry approaches, a call's payoff as a function of strike develops a sharper **kink** (the option value hits exactly zero for large strikes). Capturing this increasingly non-linear payoff shape requires **more** mixture components. Conversely, options far from expiry have an almost **straight-line** payoff-versus-strike relationship, which a handful of components can already capture well. This is why short-dated options favour large mixtures (30) while longer-dated options are well served by a moderate mixture (8).

5.5 Results: Performance Tables VI

Qualitative Picture From Figures 5.1–5.5

- Across all five maturities, predicted option prices from every mixture size visually track the market data closely as a function of strike (a smooth, decreasing, convex curve, as required by Eqs. 5.1–5.2).
- However, no model fits the market **perfectly**: the absolute error plots show non-zero errors throughout.
- **Errors are largest at small strikes** (where option prices themselves are large – deep in-the-money) and **smallest at large strikes** (where option prices approach zero – deep out-of-the-money). This is a consistent pattern across all maturities and mixture sizes.

Practical Takeaway

Because the risk-neutral density is modelled directly, and the pricing function is derived from it via Eq. (5.5), every model configuration studied here is **automatically arbitrage-free** – no custom penalty terms were needed to enforce Eqs. (5.1)–(5.4), unlike direct neural-network regression of $f_w(K)$.

5.6 Conclusions I

Summary

- This chapter analyzed JSE All Share Index option prices across five maturities and many strikes using a **mixture-of-normalizing-flows** model of the risk-neutral density.
- Modelling the density directly (rather than the price curve) **guarantees** the no-arbitrage conditions of Carr et al. (2003) are satisfied.
- The fitted mixtures track observed market prices well across all maturities and strikes.
- **Largest errors** occur at **small strikes**; **smallest errors** occur at **large strikes**.
- The **best number of mixture components depends on time to expiry**: more components help for options close to maturity (sharper payoff kink), fewer components suffice for options far from maturity (near-linear payoff).
- A single normalizing flow ($M = 1$) consistently **underperforms** any mixture.

5.6 Conclusions II

Open Directions Noted by the Authors

- Developing a principled method to **select the optimal number of mixture components**, rather than trying a fixed grid $\{1, 8, 15, 30\}$.
- **Jointly** modelling the risk-neutral density across *all* maturities and strikes simultaneously (i.e., the full option price **surface**), instead of fitting one density per expiry.

Looking Ahead

The next chapter (Chapter 6) shifts topic entirely: it models corporations' **credit ratings** using **distributed Gaussian processes** – motivated by how strongly credit ratings affect the interest rate at which an entity can borrow.